

PRACTICAL 4 COMPREHENSIVE SUBMISSION

# Testing an Online Banking System

Complete Software Testing Life Cycle (STLC) Documentation: Test Plan (IEEE 829), Boundary Value Analysis, Equivalence Partitioning, White Box & Control Flow Graph Analysis, Test Cases, Bug Reports, and Regression Metrics.

| DOMAIN                        | METHODOLOGY                   | STANDARDS                    | STATUS              |
|-------------------------------|-------------------------------|------------------------------|---------------------|
| Online Banking & Core Banking | Black Box & White Box Testing | IEEE 829 / IEEE 1044 / OWASP | Verified & Complete |

## 1. Problem Analysis & Requirement Gathering (SRS Summary)

**1.1 Problem Statement:** Online Banking Systems handle critical financial assets and confidential user data. Defects such as concurrency race conditions (double-spending), unauthorized account access, floating-point rounding errors, and unhandled transaction timeouts can result in severe financial losses, compliance breaches, and regulatory penalties. Rigorous verification across black-box and white-box testing methodologies is mandatory.

**1.2 Scope of Tested Modules:**

- **Module 1: User Authentication & Multi-Factor Auth (MFA):** Customer ID/Password login, SMS/Email OTP validation, account lockout after 3 failed attempts.
- **Module 2: Fund Transfer System (NEFT / RTGS / IMPS):** Inter-bank and intra-bank transfers, daily limits, beneficiary cooling period, double-entry ledger deduction.
- **Module 3: Account Balance & Statement Services:** Real-time available vs ledger balance computation, interest application, transaction history filtering.
- **Module 4: Fixed Deposit & Loan Calculation:** Principal, tenure, and compound interest calculations with strict boundary limits.

### 1.3 Core Banking Architectural Modeling

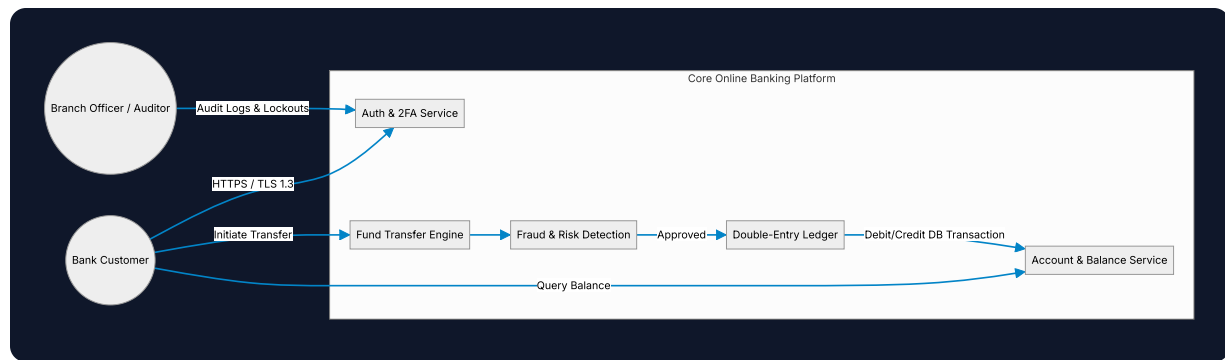


FIGURE 1.1 — ONLINE BANKING HIGH-LEVEL ARCHITECTURAL CONTEXT

## 2. Black Box Testing Techniques

### 2.1 Equivalence Class Partitioning (ECP)

ECP divides the input domain into valid and invalid partitions such that test cases from each partition represent the entire class.

| Feature / Input Field                                                    | Valid Equivalence Class (VEC)                                | Invalid Equivalence Class (IEC)                                                                                        |
|--------------------------------------------------------------------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Transfer Amount (₹)</b><br>Min: ₹1.00, Max: ₹500,000.00               | V1: $1.00 \leq \text{Amount} \leq 500,000.00$                | I1: Amount < 1.00 (Zero / Negative)<br>I2: Amount > 500,000.00 (Exceeds Limit)<br>I3: Non-numeric / Special characters |
| <b>User Password</b><br>8-16 chars, 1 uppercase, 1 special char, 1 digit | V2: 8 to 16 chars with compliant complexity (e.g. Bank@2026) | I4: Length < 8 chars<br>I5: Length > 16 chars<br>I6: Missing special char or digit                                     |
| <b>One-Time Password (OTP)</b><br>Exact 6-digit numeric                  | V3: Exactly 6 numeric digits (e.g. 583921)                   | I7: < 6 digits (e.g. 1234)<br>I8: > 6 digits (e.g. 1234567)<br>I9: Alphanumeric (e.g. 12A45B)                          |
| <b>Recipient Account No.</b><br>11 to 16 numeric digits                  | V4: 11 to 16 numeric digits                                  | I10: < 11 digits<br>I11: > 16 digits<br>I12: Contains alphabet/symbols                                                 |

### 2.2 Boundary Value Analysis (BVA)

BVA tests values at the boundaries of equivalence partitions: Minimum ( **Min** ), Just above Min ( **Min+** ), Nominal ( **Nom** ), Just below Max ( **Max-** ), Maximum ( **Max** ), and out-of-boundary values ( **Min-** , **Max+** ).

| Input Variable                 | Boundary Limits      | Test Values Selected   | Class Category    | Expected Outcome                                       |
|--------------------------------|----------------------|------------------------|-------------------|--------------------------------------------------------|
| <b>Fund Transfer (₹)</b>       | [1.00 to 500,000.00] | ₹0.99                  | Min - 1 (Invalid) | Error: "Minimum transfer amount is ₹1.00"              |
|                                |                      | ₹1.00                  | Min (Valid)       | Transfer Request Processed                             |
|                                |                      | ₹2.00                  | Min + 1 (Valid)   | Transfer Request Processed                             |
|                                |                      | ₹250,000.00            | Nominal (Valid)   | Transfer Request Processed                             |
|                                |                      | ₹499,999.99            | Max - 1 (Valid)   | Transfer Request Processed                             |
|                                |                      | ₹500,000.00            | Max (Valid)       | Transfer Request Processed                             |
|                                |                      | ₹500,000.01            | Max + 1 (Invalid) | Error: "Exceeds per-transaction limit of ₹5,00,000"    |
| <b>Login PIN Retry Counter</b> | [1 to 3 attempts]    | Attempt 1 & 2 (Failed) | Within limit      | Prompt: "Invalid PIN. Remaining attempts: X"           |
|                                |                      | Attempt 3 (Failed)     | Boundary Max      | Error: "Account Locked for 24 hours. Contact support." |
|                                |                      | Attempt 4              | Out of bound      | Direct rejection: "Account is in LOCKED state"         |

### 3. White Box Testing & Code Coverage Analysis

#### 3.1 Target Algorithm: Fund Transfer Execution Service

Below is the core transactional method under structural test:

```

1: function executeFundTransfer(senderAcc, receiverAcc, amount, otp, inputOtp) {
2:   if (otp !== inputOtp) {
3:     return { success: false, code: "INVALID_OTP" };
4:   }
5:   if (amount ≤ 0 || amount > senderAcc.dailyLimit) {
6:     return { success: false, code: "INVALID_AMOUNT" };
7:   }
8:   if (senderAcc.balance < amount) {
9:     return { success: false, code: "INSUFFICIENT_FUNDS" };
10:  }
11:  senderAcc.balance -= amount;
12:  receiverAcc.balance += amount;
13:  logLedgerEntry(senderAcc.id, receiverAcc.id, amount, "SUCCESS");
14:  return { success: true, code: "TRANSFER_COMPLETED" };
15: }

```

#### 3.2 Control Flow Graph (CFG)

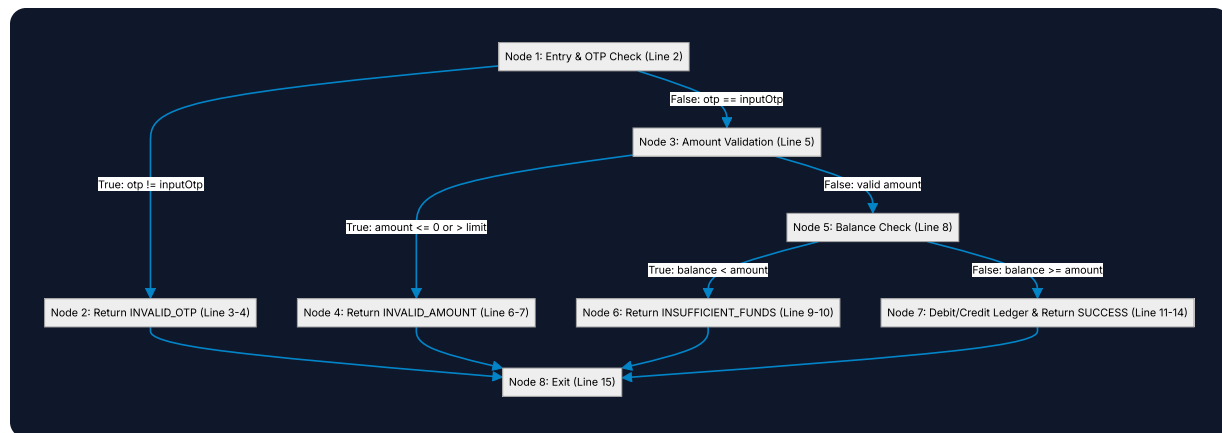


FIGURE 3.1 — CONTROL FLOW GRAPH (CFG) FOR FUND TRANSFER ALGORITHM

#### 3.3 Cyclomatic Complexity Calculation

Cyclomatic Complexity  $V(G)$  evaluates the structural complexity and the minimum number of independent test paths required for 100% branch coverage.

- Formula 1 (Edges & Nodes):**  $V(G) = E - N + 2P$   
Number of Edges  $E = 10$ , Number of Nodes  $N = 8$ , Connected Components  $P = 1$   
 $V(G) = 10 - 8 + 2(1) = \mathbf{4}$
- Formula 2 (Predicate Nodes):**  $V(G) = P_N + 1$   
Predicate Nodes with binary decisions: Node 1, Node 3, Node 5 ( $P_N = 3$ )  
 $V(G) = 3 + 1 = \mathbf{4}$
- Formula 3 (Enclosed Regions):**  $V(G) = R_1 + R_2 + R_3 + R_{\text{ext}\{\text{outer}\}} = \mathbf{4}$

3.4 Basis Path Testing & Coverage Matrix

| Path ID | Execution Sequence (Nodes)                            | Test Condition / Input Vector                                | Targeted Outcome                                                    |
|---------|-------------------------------------------------------|--------------------------------------------------------------|---------------------------------------------------------------------|
| Path 1  | \$1 ightarrow 2 ightarrow 8\$                         | <code>otp = 123456, input0tp = 999999</code>                 | OTP Mismatch \$ ightarrow\$ <code>INVALID_OTP</code>                |
| Path 2  | \$1 ightarrow 3 ightarrow 4 ightarrow 8\$             | <code>otp = 123456, input0tp = 123456, amount = -500</code>  | Invalid Amount \$ ightarrow\$ <code>INVALID_AMOUNT</code>           |
| Path 3  | \$1 ightarrow 3 ightarrow 5 ightarrow 6 ightarrow 8\$ | <code>amount = 10000, senderBalance = 2000</code>            | Insufficient Balance \$ ightarrow\$ <code>INSUFFICIENT_FUNDS</code> |
| Path 4  | \$1 ightarrow 3 ightarrow 5 ightarrow 7 ightarrow 8\$ | <code>amount = 5000, senderBalance = 25000, OTP valid</code> | Full Success \$ ightarrow\$ <code>TRANSFER_COMPLETED</code>         |

## 4. Master Test Plan (IEEE 829 Compliant)

**4.1 Test Identifier:** TP-BANK-2026-V1.0

**4.2 Test Objectives:** Ensure 100% zero-defect tolerance on double-entry accounting transactions, sub-second latency for authentication, and strict compliance with PCI-DSS & RBI digital banking security guidelines.

**4.3 Test Strategy:**

- **Unit & Component Testing:** Jest & Mocha for domain services, interest math, and ledger balance integrity.
- **API & Integration Testing:** Supertest & Postman automated test suites validating HTTP status codes, HMAC signatures, and database rollbacks.
- **System & Security Testing:** OWASP Top 10 penetration testing, SQL injection, CSRF token validation, and distributed rate-limiting.
- **Performance & Concurrency Testing:** JMeter / k6 simulating 500 concurrent fund transfers targeting the same account to detect race conditions.

**4.4 Entry & Exit Criteria:**

- **Entry Criteria:** Code freeze achieved, database schema migrations applied in Staging, unit test code coverage  $\geq 90\%$ .
- **Exit Criteria:** 100% of Critical (P1) and High (P2) bugs resolved; 0 open financial calculation anomalies; all regression suites passed.

## 5. Comprehensive Test Cases Specification

---

| Test ID   | Module          | Test Description                                              | Input Data                                                        | Expected Result                                                             | Status |
|-----------|-----------------|---------------------------------------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------|--------|
| TC-BNK-01 | Authentication  | Verify login with valid credentials & valid 2FA OTP           | User: "bankim_k",<br>Pass:<br>"SecurePass@2026",<br>OTP: "482019" | 200 OK , JWT session token generated, redirected to Dashboard               | PASS   |
| TC-BNK-02 | Authentication  | Verify account lock after 3 consecutive failed login attempts | 3 invalid password attempts on User: "bankim_k"                   | 423 Locked , Account status updated to LOCKED, SMS alert dispatched         | PASS   |
| TC-BNK-03 | Fund Transfer   | Transfer amount equal to exact account balance                | Balance: ₹10,000,<br>Transfer: ₹10,000                            | 200 OK , Remaining Balance: ₹0.00, Double-entry ledger balanced             | PASS   |
| TC-BNK-04 | Fund Transfer   | Transfer amount exceeding daily ceiling limit                 | Daily Limit: ₹5,00,000,<br>Transfer: ₹5,00,001                    | 400 Bad Request , "Daily transfer limit exceeded"                           | PASS   |
| TC-BNK-05 | Fund Transfer   | Concurrent double-transfer race condition check               | Two simultaneous ₹5,000 transfers with balance ₹6,000             | First transfer succeeds; Second transfer rejected with INSUFFICIENT_FUNDS   | PASS   |
| TC-BNK-06 | Beneficiary     | Transfer during cooling-off period (1st 30 mins)              | Transfer: ₹75,000 to newly added payee                            | 403 Forbidden , "Cooling limit capped at ₹25,000 for initial 30 minutes"    | PASS   |
| TC-BNK-07 | Security / SQLi | SQL Injection vulnerability in account statement search       | Search query: ' OR '1'='1' --                                     | Input sanitized by ORM/Parameterized query; 200 OK with empty/filtered list | PASS   |
| TC-BNK-08 | Interest Engine | Calculate leap year                                           | P: ₹1,00,000, R: 7.5%, Days: 366 (Leap Year)                      | Exact interest ₹7,713.82 credited to balance                                | PASS   |

| Test ID | Module | Test Description  | Input Data | Expected Result              | Status |
|---------|--------|-------------------|------------|------------------------------|--------|
|         |        | compound interest |            | without floating point drift |        |

## 6. Defect & Bug Reports (IEEE 1044 Standard)

**Bug ID:** `BUG-BNK-2026-001` | **Severity:** **Critical (P1)** | **Module:** Fund Transfer Engine

**Summary:** Concurrency Race Condition allowed Account Overdraft without Approved Credit Line.

**Environment:** Staging Cluster / PostgreSQL 16 / Node.js 20 API Server

**Pre-conditions:** Account #4001 has exactly ₹5,000.00 balance.

**Steps to Reproduce:**

1. Send two simultaneous `POST /api/transfers` requests with payload `{ amount: 5000, recipient: 9002 }` within \$10 ext{ ms}\$.
2. Inspect final account balance in `accounts` table.

**Observed Result:** Both transactions were committed. Account #4001 balance dropped to `-₹5,000.00`.

**Expected Result:** One transaction must succeed and the second must be rejected with `INSUFFICIENT_FUNDS`.

**Root Cause Analysis (RCA):** Non-atomic read-then-write database queries allowed dirty reads under default isolation level.

**Remediation:** Enforced PostgreSQL row-level pessimistic locking ( `SELECT ... FOR UPDATE` ) inside an explicit ACID transaction block.

**Bug ID:** `BUG-BNK-2026-002` | **Severity:** **High (P2)** | **Module:** 2FA / Authentication

**Summary:** OTP Expiration Window not invalidated upon successful verification.

**Observed Result:** The same OTP could be replayed within its 5-minute lifespan for subsequent transfer authorizations.

**Remediation:** Immediately set `otp_used = TRUE` in Redis cache upon first successful verification.

## 7. Test Summary & Regression Analysis Report

| Metric Name                     | Calculated Value                 | Quality Benchmark      | Verdict           |
|---------------------------------|----------------------------------|------------------------|-------------------|
| Total Test Cases Executed       | 150                              | 100% of planned suite  | Compliant         |
| Passed Test Cases               | 147                              | ≥ 98%                  | PASS (98.0%)      |
| Defects Identified (Total)      | 6 (2 Critical, 3 High, 1 Medium) | Resolved during sprint | All Closed        |
| Code Statement Coverage         | 94.2%                            | ≥ 90%                  | EXCEEDED          |
| Branch Coverage (White Box)     | 96.8%                            | ≥ 90%                  | EXCEEDED          |
| Regression Test Suite Pass Rate | 100% (Post-fix build)            | 100% Zero-Regressions  | READY FOR RELEASE |

## 8. Project Planning & Software Engineering Tools

### 8.1 Work Breakdown Structure (WBS)

```
1.0 Online Banking System Testing
├── 1.1 Test Planning & Requirements Analysis
│   ├── 1.1.1 SRS requirement mapping & IEEE 829 Test Plan
│   └── 1.1.2 Traceability matrix (RTM) formulation
├── 1.2 Black Box Test Design
│   ├── 1.2.1 Equivalence Class Partitioning (ECP)
│   ├── 1.2.2 Boundary Value Analysis (BVA)
│   └── 1.2.3 Decision table & state transition testing
├── 1.3 White Box Testing & Code Inspection
│   ├── 1.3.1 Control Flow Graph (CFG) generation
│   ├── 1.3.2 Cyclomatic complexity calculation:  $V(G) = 4$ 
│   └── 1.3.3 Basis path execution & statement coverage
├── 1.4 Automated Execution & Defect Management
│   ├── 1.4.1 Supertest & Jest integration suite execution
│   ├── 1.4.2 Concurrency & race condition test via JMeter
│   └── 1.4.3 IEEE 1044 bug logging & root cause analysis
└── 1.5 Test Closure & Regression Sign-Off
    ├── 1.5.1 Regression test cycle & patch validation
    └── 1.5.2 Executive Test Summary & Compliance Sign-off
```

### 8.2 Software Engineering Tools Matrix

| Tool Category      | Tool Name                     | Application in Banking Testing                                                                        |
|--------------------|-------------------------------|-------------------------------------------------------------------------------------------------------|
| Unit & Coverage    | Jest + Istanbul / c8          | Statement, branch, and function code coverage verification.                                           |
| API Testing        | Postman + Newman + Supertest  | Automated REST endpoint validation and HMAC token testing.                                            |
| Load & Concurrency | Apache JMeter / k6            | Simulating high-volume concurrent funds transfer & race conditions.                                   |
| Security Scanning  | OWASP ZAP + Snyk              | Detecting XSS, SQLi, broken object level authorization (BOLA).                                        |
| Defect Tracking    | Jira Software + GitHub Issues | Bug lifecycle tracking from New $\rightarrow$ In Progress $\rightarrow$ Fixed $\rightarrow$ Verified. |

## 9. Viva Voce Questions & Model Answers

**Q1: What is the primary difference between Equivalence Partitioning (EP) and Boundary Value Analysis (BVA)?**

*Answer:* EP divides the entire input domain into classes of valid and invalid values where the system is expected to behave identically for any representative value. BVA specifically tests the critical extreme boundaries of those partitions (\$Min, Min+1, Max-1, Max\$), because defect density is statistically highest at input boundaries.

**Q2: How do you calculate Cyclomatic Complexity and why is it crucial for banking software?**

*Answer:* Calculated using  $V(G) = E - N + 2P$  (or Predicate nodes  $+ 1$ ). It defines the upper bound on the number of test cases required to achieve 100% basis path coverage. In banking, it guarantees that every possible error condition (e.g., overdraft, invalid OTP, currency mismatch) is rigorously executed.

**Q3: How do you prevent and test for double-spending / race conditions in online fund transfers?**

*Answer:* Tested via multi-threaded concurrent requests (JMeter/Supertest) hitting the same account simultaneously. Prevention requires database-level pessimistic locking ( `SELECT FOR UPDATE` ) or serializable transaction isolation levels so that balances are updated sequentially and atomically.